Name : Ashwin Rai Roll No. : 202403069 Class : TYCS - A

Practical No. 4: Digital Signature using RSA

Aim

To implement the RSA Digital Signature algorithm for signing and verifying messages to ensure their integrity and authenticity.

Objectives

- To understand the concept of Digital Signatures.
- To study the RSA Digital Signature algorithm.
- To generate RSA public and private keys.
- To create a digital signature using the private key.
- To verify the authenticity and integrity of a digitally signed message using the public key.

Theory

1. Digital Signature

- A **Digital Signature** is a cryptographic technique used to verify the **authenticity**, **integrity**, and **non-repudiation** of a digital message or document.
- Unlike encryption, which provides confidentiality, a digital signature proves that the message was created by the owner of the private key and has not been modified during transmission.

Digital signatures use **asymmetric cryptography**, where:

- **Private Key** – Used to create the digital signature.
- **Public Key** – Used to verify the digital signature.

2. RSA Digital Signature Algorithm

- RSA (Rivest–Shamir–Adleman) is one of the most widely used public-key cryptographic algorithms. It uses a pair of mathematically related keys.

Before signing a message:

- A cryptographic hash (SHA-256) of the message is generated.
- The hash is encrypted using the sender's private key.
- The encrypted hash becomes the **Digital Signature**.

During verification:

- The receiver computes the hash of the received message.
- The received signature is verified using the sender's public key.
- If both hash values match, the signature is valid.

Digital Signature Generation

- The sender creates a digital signature using the private key.

Steps

1. Generate RSA public and private keys.
2. Read the message.
3. Compute the SHA-256 hash of the message.
4. Encrypt the hash using the private key.
5. Generate the Digital Signature.

Digital Signature Verification

The receiver verifies the digital signature using the public key.

Steps

1. Receive the message and digital signature.
2. Compute the SHA-256 hash of the received message.
3. Verify the signature using the sender's public key.
4. If verification succeeds, the message is authentic and unchanged.

Algorithm

A. Digital Signature Generation

1. Generate the RSA public and private keys.
2. Read the message from the user.
3. Compute the SHA-256 hash of the message.
4. Create the digital signature using the private key.
5. Display the generated digital signature.

B. Digital Signature Verification

1. Read the digital signature entered by the user.
2. Verify the signature using the public key.
3. If verification is successful, display **"Digital Signature is Valid."**
4. Otherwise, display **"Digital Signature is Invalid."**

Program: RSA Digital Signature

```
import hashlib
import rsa

public_key, private_key = rsa.newkeys(512)

def create_signature(private_key, message):
    message_hash = hashlib.sha256(message.encode()).digest()
    signature = rsa.sign(message_hash, private_key, 'SHA-256')
    return signature

def verify_signature(public_key, message, signature):
    message_hash = hashlib.sha256(message.encode()).digest()

    try:
        rsa.verify(message_hash, signature, public_key)
        return True
    except rsa.VerificationError:
        return False


if __name__ == "__main__":

    message = input("Enter the Message: ")

    signature = create_signature(private_key, message)

    print("\nDigital Signature:")
    print(signature.hex())

    choice = input("\nDo you want to Verify the Signature? (yes/no): ")

    if choice.lower() == "yes":
```

```
    received_signature = input("Enter the Digital Signature: ")

    if verify_signature(public_key, message, bytes.fromhex(received_signature)):
        print("Digital Signature is Valid. The message is Authentic.")
    else:
        print("Digital Signature is Invalid. The message may have been Tampered.")

else:
    print("Verification Skipped.")
```

Sample Output

```
========== RESTART: C:/Users/CC2_50/Documents/CIS_Prac4.py ==========
Enter the Message: Hello

Digital Signature:
35b6ddf01184ecce96bd1a9455fde287b5be9253a44db90d48e2f81dca657754a25e33fa59af80f330097f60d127701b854a80d7eecdb326f7fa97bed247ff78

Do you want to Verify the Signature? (yes/no): no
Verification Skipped.
```